

KDD Cup 1999

Autoencoder Anomaly Detection

Professional presentation summary of the notebook workflow, charts, metrics, and final recommendation

Final Winner

Autoencoder

Higher recall and F1-score, with far fewer missed attacks than Isolation

Forest. Missed attacks	125
---------------------------	------------

F1-score	0.9931
----------	---------------

Recall	0.9984
--------	---------------

Prepared by Raturaj Mokashi

Generated from uploaded notebook: 4_kdd_cup_autoencoder.ipynb

Executive Summary

DATASET SIZE

494,021

network connections

PROCESSED FEATURES

74

after scaling + encoding

AUTOENCODER F1

0.9931

best overall score

MISSED ATTACKS

125

vs 773 for Isolation Forest

What this work proves

- The KDD dataset was converted from many attack labels into a binary Normal vs Anomaly target.
- The Autoencoder learned normal network traffic only, then used reconstruction error as the anomaly score.
- The 95th percentile of normal training reconstruction error was used as the threshold.
- Against Isolation Forest, the Autoencoder produced stronger recall and F1-score and missed far fewer attacks.

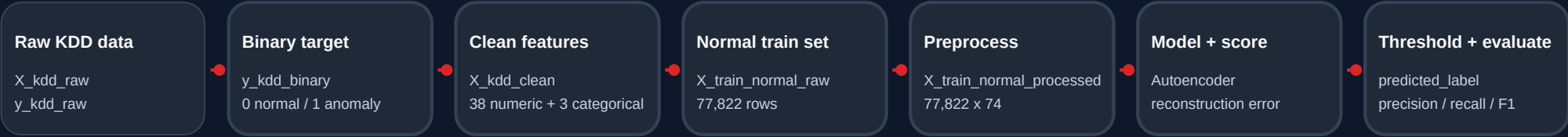
Recommendation

Use Autoencoder as the preferred KDD anomaly detection model because missed attacks are the key security risk.

Autoencoder

Best recall + F1-score

End-to-End Workflow



Key variables and why they matter

Variable	Created from	Purpose
X_kdd_clean	X_kdd_raw	Feature types corrected before preprocessing
y_kdd_binary	y_kdd_raw	Binary label: 0 normal, 1 anomaly
X_train_normal_processed	X_train_normal_raw	Training input for Autoencoder
X_test_processed	X_test_raw	Final test input for predictions
reconstruction_results	X_test_processed + y_test	Stores error, threshold prediction, and metrics

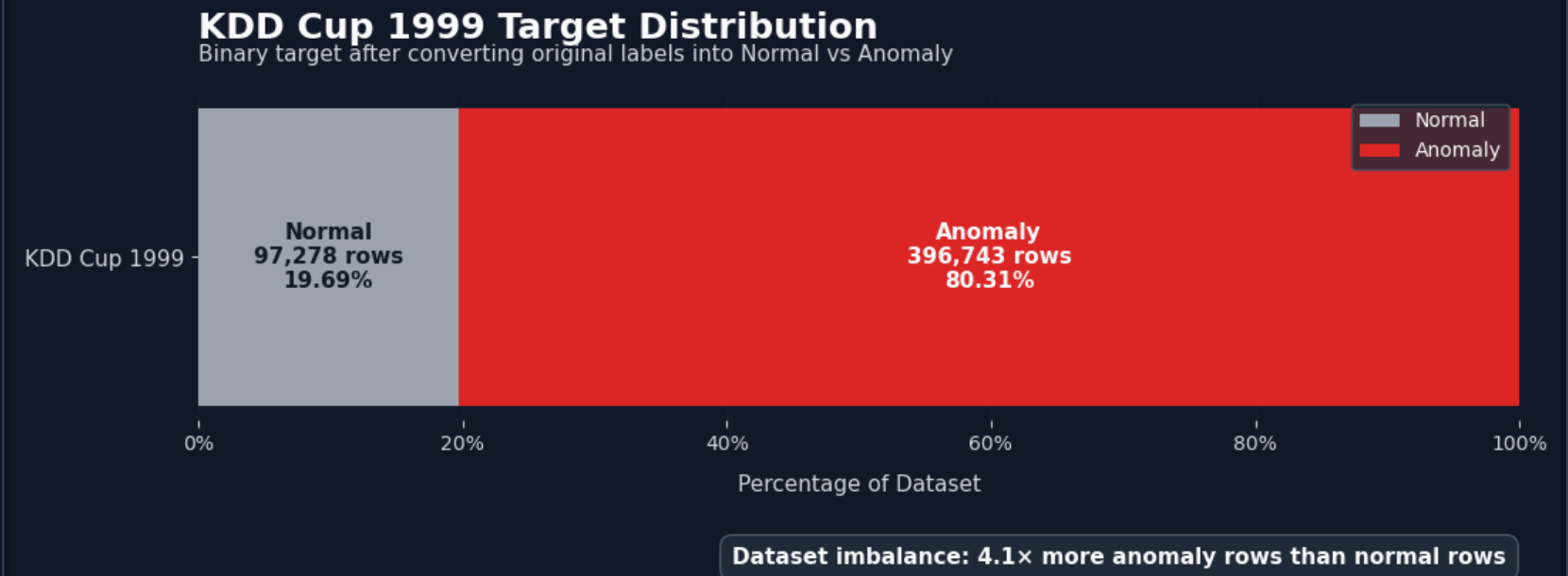
Target Distribution: Attack-Heavy Dataset

Story

- The original KDD labels were converted into Normal vs Anomaly.
- Anomalies form 80.31% of the dataset, while normal rows are 19.69%.
- The Autoencoder still trains on normal rows only to learn a clean normal baseline.

READOUT

The dataset is not balanced: there are 4.1x more anomaly rows than normal rows.

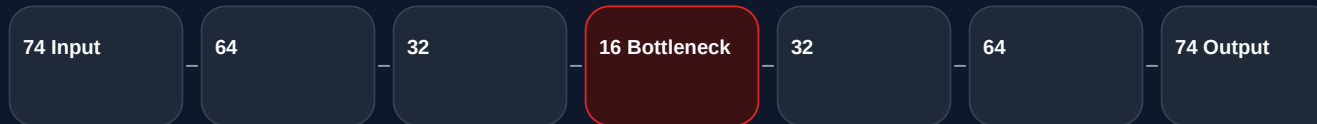


Preprocessing and Autoencoder Architecture

Preprocessing

Feature group	Count	Method
Numerical columns	38	Median imputation + StandardScaler
Categorical columns	3	Most-frequent imputation + OneHotEncoder
Final feature count	74	Numeric input for neural network

Autoencoder shape



Why this architecture?

- The model compresses 74 processed features through a 16-neuron bottleneck.
- This forces the network to learn core normal-traffic patterns.
- Rows that do not match normal behavior reconstruct poorly and receive high error.
- The model had 14,874 trainable parameters.

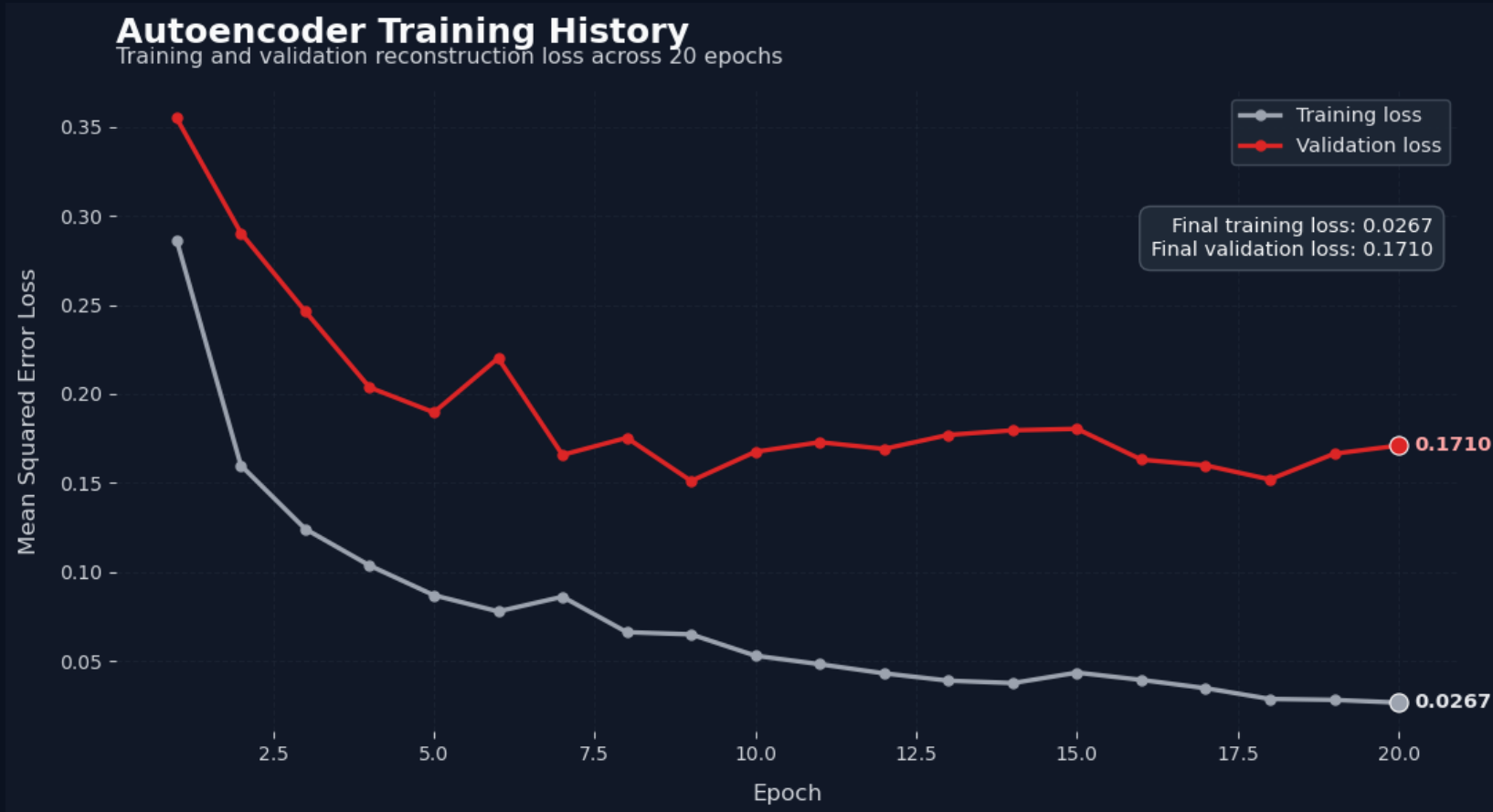
Training History: Model Learns Normal Traffic

Read this chart

- Training loss drops from the first epoch to a final value of 0.0267.
- Validation loss ends at 0.1710 and fluctuates after the early drop.
- The model is learning to reconstruct normal training rows over 20 epochs.

READOUT

The model learned normal patterns well enough to use reconstruction error for anomaly detection.



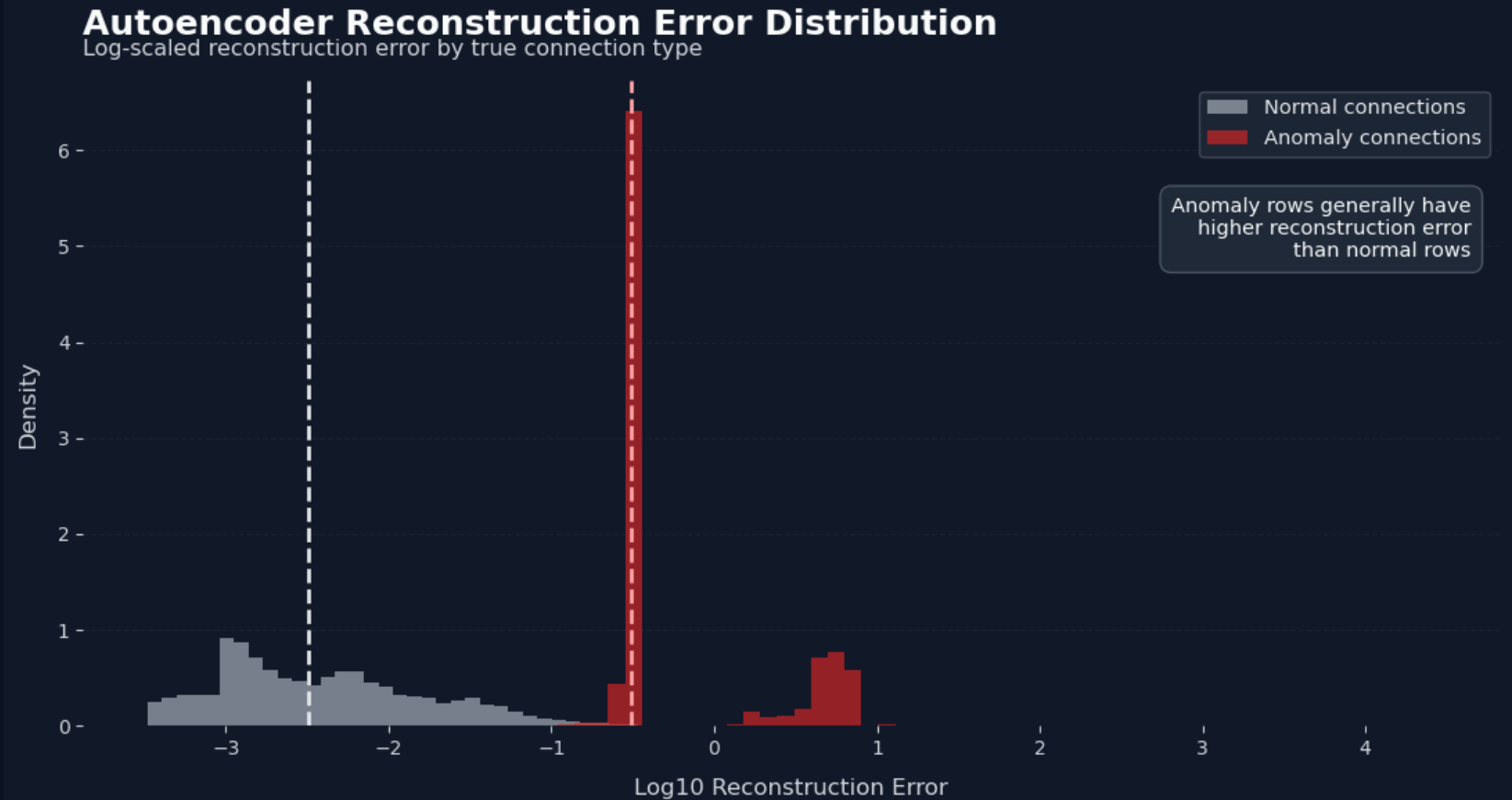
Reconstruction Error Separates Normal and Anomaly Rows

Read this chart

- Normal rows cluster on the lower-error side.
- Anomaly rows are shifted to higher reconstruction error.
- Median normal error is 0.0032 versus 0.3075 for anomalies.

READOUT

Reconstruction error becomes the anomaly score: higher error means more suspicious.



Threshold Strategy and Prediction Output

Threshold method

The anomaly threshold was set as the 95th percentile of normal training reconstruction error.

```
if reconstruction_error > anomaly_threshold:
    predicted_label = 1 # anomaly
else:
    predicted_label = 0 # normal
```

THRESHOLD

0.056505

95th percentile of normal error

PREDICTED ANOMALY

80,198

81.17% of test set

Prediction	Rows	Percent
Predicted Anomaly	80,198	81.17%
Predicted Normal	18,607	18.83%

Prediction output closely matches the test distribution, which is a good sign before formal evaluation.

Autoencoder Confusion Matrix

Evaluation result

- Correctly detected 79,224 anomaly rows.
- Missed only 125 attack rows.
- Produced 974 false alarms among normal traffic.
- High recall is critical in network security because missed attacks are dangerous.

READOUT

Recall = 0.9984. The Autoencoder caught almost all actual attacks.

Autoencoder Confusion Matrix

Evaluation of predicted anomalies against true KDD test labels

Actual Label	Predicted Label	
	Predicted Normal	Predicted Anomaly
Actual Normal	18,482 Correct Normal	974 False Alarm
Actual Anomaly	125 Missed Attack	79,224 Correct Attack

Security focus:
Minimize missed attacks.
This model missed only
125 anomaly rows.

Isolation Forest Baseline

Why compare with Isolation Forest?

- It is a traditional tree-based anomaly detection algorithm.
- To make the comparison fair, it was trained on normal processed rows only.
- It used the same 95th percentile threshold idea on normal training scores.
- This created a strong baseline against the neural-network Autoencoder.

IF THRESHOLD

0.506717

normal-score percentile

IF F1-SCORE

0.9894

strong baseline

Outcome	Rows
Correct Normal	18,546
False Alarm	910
Missed Attack	773
Correct Attack	78,576

Isolation Forest performed well, but missed 773 attacks - substantially more than the Autoencoder.

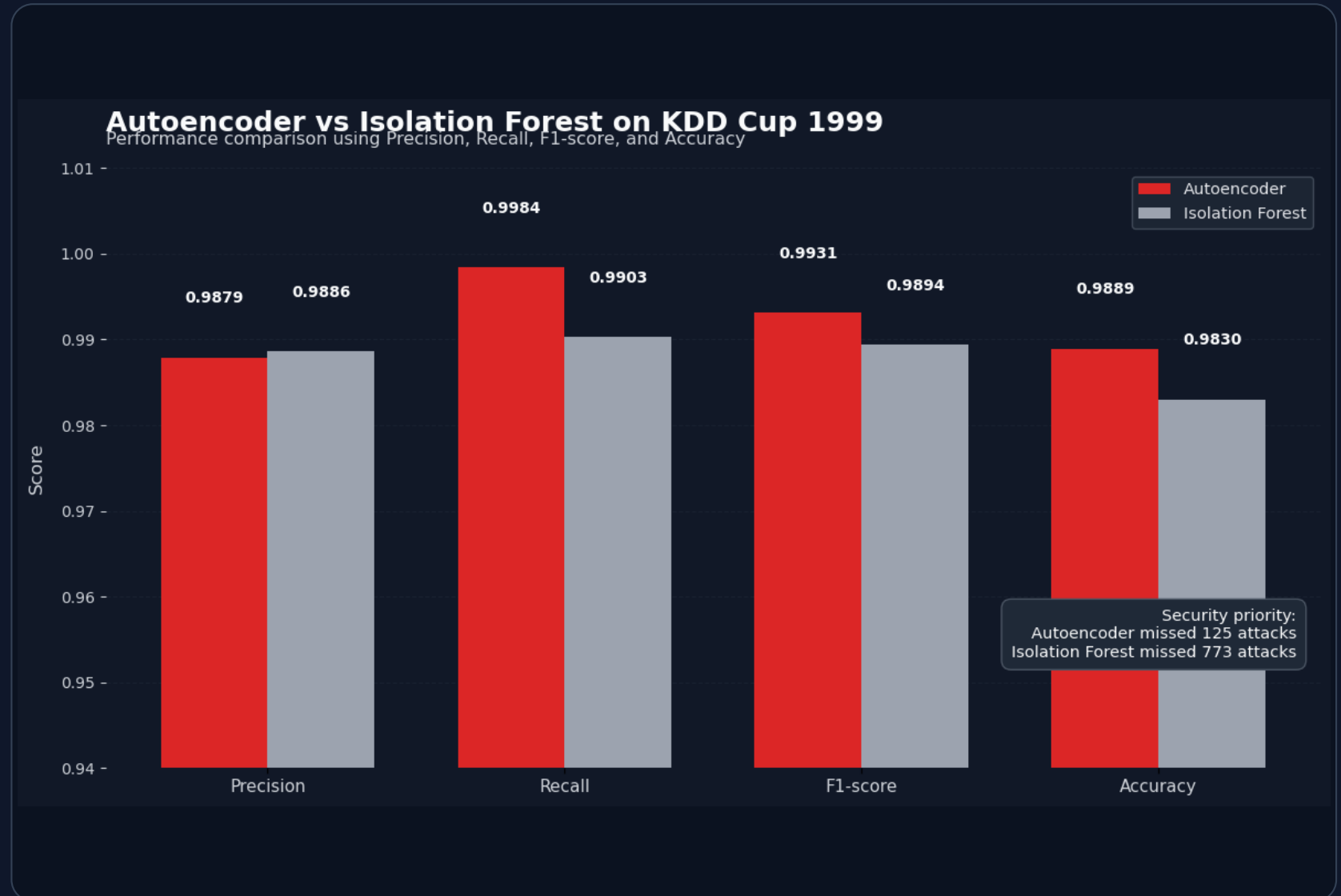
Metric Comparison: Autoencoder vs Isolation Forest

Model leaderboard

- Isolation Forest has slightly higher precision.
- Autoencoder has higher recall, F1-score, and accuracy.
- For security, recall and missed attacks matter more than tiny precision differences.

READOUT

Autoencoder wins on F1-score and recall, the most important security metric in this task.



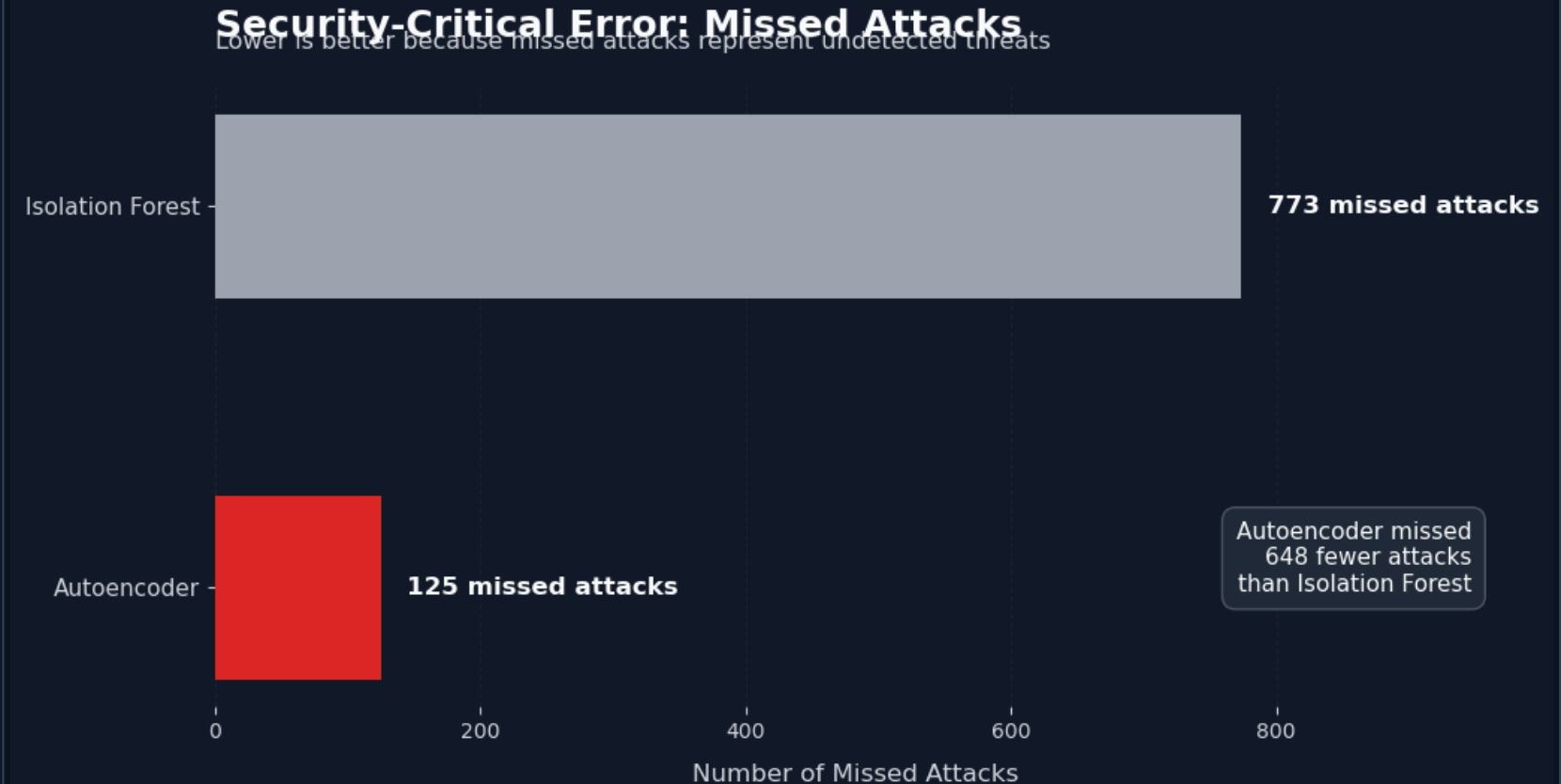
Security-Critical Error: Missed Attacks

Business impact

- False alarms are inconvenient but manageable.
- Missed attacks are dangerous because threats pass through undetected.
- Autoencoder missed 125 attacks; Isolation Forest missed 773.
- That is 648 fewer missed attacks for the Autoencoder.

READOUT

The clearest business case: Autoencoder misses far fewer attacks.



Final Recommendation

Use Autoencoder as the primary model

- Best F1-score: 0.9931
- Best recall: 0.9984
- Only 125 missed attacks
- Reconstruction error gave a clear separation between normal and anomaly rows
- Better suited for security monitoring where missed threats are costly

Executive conclusion

The Autoencoder learned normal network behavior from clean normal rows, converted reconstruction error into an anomaly score, and outperformed Isolation Forest on the security-critical objective: reducing missed attacks.

Autoencoder wins

125 missed attacks vs 773

648 fewer missed attacks than Isolation Forest